

RESTler: Stateful REST API Fuzzing

Vaggelis Atlidakis*
Columbia University

Patrice Godefroid
Microsoft Research

Marina Polishchuk
Microsoft Research

Abstract—This paper introduces *RESTler*, the first stateful REST API fuzzer. *RESTler* analyzes the API specification of a cloud service and generates sequences of requests that automatically test the service through its API. *RESTler* generates test sequences by (1) inferring *producer-consumer dependencies* among *request types* declared in the specification (e.g., inferring that “a request B should be executed after request A” because B takes as an input a resource-id x produced by A) and by (2) analyzing *dynamic feedback* from responses observed during prior test executions in order to generate new tests (e.g., learning that “a request C after a request sequence A;B is refused by the service” and therefore avoiding this combination in the future).

We present experimental results showing that these two techniques are necessary to thoroughly exercise a service under test while pruning the large search space of possible request sequences. We used *RESTler* to test GitLab, an open-source Git service, as well as several Microsoft Azure and Office365 cloud services. *RESTler* found 28 bugs in GitLab and several bugs in each of the Azure and Office365 cloud services tested so far. These bugs have been confirmed and fixed by the service owners.

I. INTRODUCTION

Over the last decade, we have seen an explosion in cloud services for hosting software applications (Software-as-a-Service), for building distributed services and data processing (Platform-as-a-Service), and for providing general computing infrastructure (Infrastructure-as-a-Service). Today, most cloud services, such as those provided by Amazon Web Services (AWS) [2] and Microsoft Azure [29], are programmatically accessed through REST APIs [11] by third-party applications [1] and other services [31]. Meanwhile, Swagger (recently renamed OpenAPI) [40] has arguably become the most popular interface-description language for REST APIs. A Swagger specification describes how to access a cloud service through its REST API, including what requests the service can handle, what responses may be received, and the response format.

Tools for automatically testing cloud services via their REST APIs and checking whether those services are reliable and secure are still in their infancy. The most sophisticated testing tools currently available for REST APIs capture live API traffic, and then parse, fuzz and replay the traffic with the hope of finding bugs [4], [34], [7], [41], [3]. Many of these tools were born as extensions of more established website testing and scanning tools (see Section VIII). Since these REST API testing tools are all recent and not yet widely used, it is still largely unknown how effective they are in finding bugs and how security-critical those bugs are.

In this paper, we introduce *RESTler*, the first automatic stateful REST API fuzzing tool. Fuzzing [39] means automatic

test generation and execution with the goal of finding security vulnerabilities. Unlike other REST API testing tools, *RESTler* performs a lightweight static analysis of an entire Swagger specification, and then generates and executes tests that exercise the corresponding cloud service in a stateful manner. By *stateful*, we mean that *RESTler* attempts to explore service states that are reachable only using sequences of multiple requests. With *RESTler*, each test is defined as a sequence of requests and responses. *RESTler* generates tests by:

- 1) inferring *dependencies* among request types declared in the Swagger specification (e.g., inferring that a resource included in the response of a request A is necessary as input argument of another request B, and therefore that A should be executed before B), and by
- 2) analyzing *dynamic feedback* from responses observed during prior test executions in order to generate new tests (e.g., learning that “a request C after a request sequence A;B is refused by the service” and therefore avoiding this combination in the future).

We present empirical evidence showing that these two techniques are necessary to thoroughly test a service, while pruning the large search space defined by all possible request sequences. *RESTler* also implements several search strategies (akin to those used in model-based testing [43]) and we compare their effectiveness while fuzzing GitLab [13], an open-source self-hosted Git service with a complex REST API.

During the course of our experiments, we found 28 new bugs in GitLab (see Section VI). We also ran experiments on four public cloud services in Microsoft Azure [29] and Office365 [30] and found several bugs in each service tested (see Section VII). This paper makes the following contributions:

- We introduce *RESTler*, the first automatic, stateful fuzzing tool for REST APIs, which analyzes a Swagger specification, automatically infers dependencies among request types, and dynamically generates tests guided by feedback from service responses.
- We present detailed experimental evidence showing that the techniques used in *RESTler* are necessary for effective automated stateful REST API fuzzing.
- We present experimental results obtained with three different strategies for searching the large search space defined by all possible request sequences, and discuss their strengths and weaknesses.
- We present a detailed case study with GitLab, a large popular open-source self-hosted Git service and discuss several new bugs found so far.
- We discuss preliminary experiences using *RESTler* to test

*The work of this author was mostly done at Microsoft Research.

blog/posts : Operations related to blog posts

GET	/blog/posts/	Returns list of blog posts
POST	/blog/posts/	Creates a new blog post
DELETE	/blog/posts/{id}	Deletes a blog post with matching "id"
GET	/blog/posts/{id}	Returns a blog post with matching "id"
PUT	/blog/posts/{id}	Updates a blog post with matching "id" and "checksum"

Fig. 1: Swagger Specification of Blog Posts Service

several Microsoft public cloud services.

The remainder of the paper is organized as follows. Section II describes how Swagger specifications are processed by *RESTler*. Sections III and IV present the main test-generation algorithm used in *RESTler* and implementation details. Section V presents an evaluation of the test-generation techniques and search strategies implemented in *RESTler*. Section VI discusses new bugs found in GitLab. Section VII presents our experiences fuzzing several public cloud services. Section VIII discusses related work, and Section IX concludes the paper.

II. PROCESSING API SPECIFICATIONS

In this paper, we consider services accessible through REST APIs described with a Swagger specification. A client program can send messages, called *requests*, to a service and receive messages back, called *responses*. Such messages are sent over the HTTP protocol. A Swagger specification describes how to access a service through its REST API (e.g., what requests the service can handle and what responses may be expected). Given a Swagger specification, open-source Swagger tools can automatically generate a web UI that allows users to view the documentation and interact with the API via a web browser.

A sample Swagger specification, in web-UI form, is shown in Figure 1. This specification describes the API of a simple blog posts hosting service. The API consists of five *request types*, specifying the endpoint, method, and required parameters. This service allows users to create, access, update, and delete blog posts. In a web browser, clicking on any of these five request types expands the description of the request type.

For instance, selecting the second (POST) request, reveals text similar to the left of Figure 2. This text is in YAML format and describes the exact syntax expected for that specific request and its response. In this case, the *definition* part of the specification indicates that an object named *body* of type *string* is required and that an object named *id* of type *integer* is optional (since it is not required). The *path* part of the specification describes the HTTP-syntax for this POST request as well as the format of the expected response.

From such a specification, *RESTler* automatically constructs the test-generation grammar shown on the right of Figure 2. This grammar is encoded in executable *python* code. It consists of code to generate an HTTP request, of type POST in this case, and code to process the expected response of this request. Each function *restler_static* simply appends the string it takes as argument without modifying

```
basePath: '/api'
swagger: '2.0'
definitions:
  "Blog Post":
    properties:
      body:
        type: string
      id:
        type: integer
    required:
      - body
    type: object

paths:
  "/blog/posts/":
    post:
      parameters:
        - in: body
          name: payload
          required: true
          schema:
            ref: "/definitions/Blog Post"

from restler import requests
from restler import dependencies

def parse_posts(data):
    post_id = data["id"]
    dependencies.set_var(post_id)

request = requests.Request(
    restler_static("POST"),
    restler_static("/api/blog/posts/"),
    restler_static("HTTP/1.1"),
    restler_static("{}"),
    restler_static("body:"),
    restler_fuzzable("string"),
    restler_static("{}"),
    'post_send': {
        'parser': parse_posts,
        'dependencies': [
            post_id.writer(),
        ]
    }
)
```

Fig. 2: Swagger Specification and Automatically Derived *RESTler* Grammar. Shows a snippet of Swagger specification in YAML (left) and the corresponding grammar generated by *RESTler* (right).

it. In contrast, the function *restler_fuzzable* takes as argument a value type (like *string* in this example) and replaces it by one value of that type taken from a (small) *dictionary* of values for that type. How dictionaries are defined and how values are selected is discussed in the next section.

The response is expected to return a new *dynamic object* (a dynamically created resource id) named *id* of type *integer*. Using the schema shown on the left, *RESTler* automatically generates the function *parse_posts* shown on the right.

By similarly analyzing the other request types described in this Swagger specification, *RESTler* will infer automatically that *ids* returned by such POST requests are necessary to generate well-formed requests of the last three request types shown in Figure 1, which each requires an *id*. These *producer-consumer dependencies* are extracted by *RESTler* when processing the Swagger specification and are later used for test generation, as described next.

III. TEST GENERATION ALGORITHM

The main algorithm for test generation used by *RESTler* is shown in Figure 3 in *python*-like notation. It starts (line 3) by processing a Swagger specification as discussed in the previous section. The result of this processing is a set of request types, denoted *reqSet* in Figure 3, and of their dependencies (more on this later).

The algorithm computes a set of request sequences, as inferred from Swagger, denoted *seqSet* and initially containing an empty sequence ϵ (line 5). A request sequence is *valid* if every response in the sequence has a valid return code, defined here as any code in the 200 range. At each iteration of its main loop (line 8), starting with $n = 1$, the algorithm computes all valid request sequences *seqSet* of length n before moving to $n + 1$ and so on until a user-specified *maxLength* is reached. Computing *seqSet* is done in two steps.

First, the set of valid request sequences of length $n - 1$ is *extended* (line 9) to create a set of new sequences of length n

```

1 Inputs: swagger_spec, maxLength
2 # Set of requests parsed from the Swagger API spec
3 reqSet = PROCESS(swagger_spec)
4 # Set of request sequences (initially an empty sequence  $\epsilon$ )
5 seqSet = { $\epsilon$ }
6 # Main loop: iterate up to a given maximum sequence length
7 n = 1
8 while (n <= maxLength):
9     seqSet = EXTEND(seqSet, reqSet)
10    seqSet = RENDER(seqSet)
11    n = n + 1
12 # Extend all sequences in seqSet by appending
13 # new requests whose dependencies are satisfied
14 def EXTEND(seqSet, reqSet):
15     newSeqSet = {}
16     for seq in seqSet:
17         for req in reqSet:
18             if DEPENDENCIES(seq, req):
19                 newSeqSet = newSeqSet + concat(seq, req)
20     return newSeqSet
21 # Concretize all newly appended requests using dictionary values,
22 # execute each new request sequence and keep the valid ones
23 def RENDER(seqSet):
24     newSeqSet = {}
25     for seq in seqSet:
26         req = last_request_in(seq)
27          $\vec{V}$  = tuple_of_fuzzable_types_in(req)
28         for  $\vec{v}$  in  $\vec{V}$ :
29             newReq = concretize(req,  $\vec{v}$ )
30             newSeq = concat(seq, newReq)
31             response = EXECUTE(newSeq)
32             if response has a valid code:
33                 newSeqSet = newSeqSet + newSeq
34             else:
35                 log error
36     return newSeqSet
37 # Check that all objects referenced in a request are produced
38 # by some response in a prior request sequence
39 def DEPENDENCIES(seq, req):
40     if CONSUMES(req)  $\subseteq$  PRODUCES(seq):
41         return True
42     else:
43         return False
44 # Objects required in a request
45 def CONSUMES(req):
46     return object_types_required_in(req)
47 # Objects produced in the responses of a sequence of requests
48 def PRODUCES(seq):
49     dynamicObjects = {}
50     for req in seq:
51         newObjs = objects_produced_in_response_of(req)
52         dynamicObjects = dynamicObjects + newObjs
53     return dynamicObjects

```

Fig. 3: Main Algorithm used in *RESTler*.

by appending each request with satisfied dependencies at the end of each sequence, as described in the `EXTEND` function (line 14). The function `DEPENDENCIES` (line 39) checks if all dependencies of the specified request are *satisfied*. This is true when every dynamic object that is a required parameter of the request, denoted by `CONSUMES(req)`, is produced by some response to the request sequence preceding it, denoted by `PRODUCES(seq)`. If all the dependencies are satisfied, the new sequence of length n is retained (line 19); otherwise it is discarded.

Second, each newly-extended request sequence whose dependencies are satisfied is *rendered* (line 10) one by one as described in the `RENDER` function (line 23). For every newly-appended request (line 26), the list of all fuzzable primitive

types in the request is computed (line 27) (those are identified by `restler_fuzzable` in the code shown on the right of Figure 2). Then, each fuzzable primitive type in the request is *concretized*, which substitutes one concrete value of that type taken out of a finite, user-configurable *dictionary* of values. For instance, for fuzzable type `integer`, *RESTler* might use a small dictionary with the values 0, 1, and -10, while for fuzzable type `string`, a dictionary could be defined with the values “sampleString”, the empty string, and a very long fixed string. The function `RENDER` generates all possible such combinations (line 28). Each combination thus corresponds to a fully-defined request `newReq` (line 29) which is HTTP-syntactically correct. The function `RENDER` then *executes* this new request sequence (line 31), and checks its response: if the response has a valid status code, the new request sequence is valid and retained (line 33); otherwise, it is discarded and the received error code is logged for analysis and debugging.

More precisely, the function `EXECUTE` executes each request in a sequence one by one, each time checking that the response is valid, extracting and memoizing dynamic objects (if any), and providing those in subsequent requests in the sequence if needed, as determined by the dependency analysis; the response returned by function `EXECUTE` in line 31 refers to the response received for the last, newly-appended request in the sequence. Note that if a request sequence produces more than one dynamic object of a given type, the function `EXECUTE` will memoize all of those objects, but will provide them later when needed by subsequent requests in the exact order in which they are produced; in other words, the function `EXECUTE` will not try different ordering of such objects. If a dynamic object is passed as argument to a subsequent request and is “destroyed” after that request, *i.e.*, it becomes unusable later on, *RESTler* will detect this by receiving an invalid status code (outside the 200 range) when attempting to reuse that unusable object, and will then discard that request sequence.

By default, the function `RENDER` of Figure 3 generates *all* possible combinations of dictionary values for every request with several fuzzable types (see line 28). For large dictionaries, this may result in astronomical numbers of combinations. In that case, a more scalable option is to randomly sample each dictionary for one (or a few) values, or to use *combinatorial-testing* algorithms [10] for covering, say, every dictionary value, or every pair of values, but not every k -tuple. In the experiments reported later, we used small dictionaries and the default `RENDER` function shown in Figure 3.

The function `EXTEND` of Figure 3 generates *all* request sequences of length $n+1$ whose dependencies are satisfied. Since n is incremented at each iteration of the main loop of line 8, the overall algorithm performs a *breadth-first search* (BFS) in the search space defined by all possible request sequences. In Section V, we report experiments performed also with two additional search strategies: `BFS-Fast` and `RandomWalk`.

BFS-Fast. In function `EXTEND`, instead of appending every request to every sequence, every request is appended to at most one sequence. This results in a smaller set `newSeqSet` which covers (*i.e.*, includes at least once) every request but

does not generate all valid request sequences. Like BFS, BFS-Fast still exercises every executable request type at each iteration of the main loop in line 8: it still provides full grammar coverage but with fewer request sequences, which allows it to go deeper faster than BFS.

RandomWalk. In function `EXTEND`, the two loops of line 17 and line 18 are eliminated; instead, the function now returns a single new request sequence whose dependencies are satisfied, and generated by *randomly* selecting one request sequence `seq` in `seqSet` and one request in `reqSet`. (The function randomly chooses such a pair until all the dependencies of that pair are satisfied.) This search strategy will therefore explore the search space of possible request sequences deeper more quickly than BFS or BFS-Fast. When RandomWalk can no longer extend the current request sequence, it restarts from scratch from an empty request sequence. (Since it does not memoize past request sequences between restarts, it might regenerate the same request sequence again in the future.)

IV. IMPLEMENTATION

We have implemented *RESTler* in 3,151 lines of modular python code split into: the parser and compiler module, the core fuzzing runtime module, and the garbage collector (GC) module. The parser and compiler module is used to parse a Swagger specification and to generate the *RESTler* grammar describing how to fuzz a target service. (In the absence of a Swagger specification, the user could directly provide the *RESTler* grammar.) The core fuzzing runtime module implements the algorithm of Figure 3 and its variants. It renders API requests, processes service-side responses to retrieve values of the dynamic objects created, and analyzes service-side feedback to decide which requests should be reused in future generations while composing new request sequences. Finally, the GC runs as a separate thread that tracks the creation of the dynamic objects over time and periodically deletes aging objects that exceed some user-defined limit (see Section VII).

A. Using RESTler

RESTler is a command-line tool that takes as input a Swagger specification, service access parameters (e.g. IP, port, authentication), the mutations dictionary, and the search strategy to use during fuzzing. After compiling the Swagger specification, *RESTler* displays the number of endpoints discovered and the list of resolved and unresolved dependencies, if any. In case of unresolved dependencies, the user may provide additional annotations or resource-specific mutations (see Section VII) and re-run this step to resolve them. Alternatively, the user may choose to start fuzzing right away and *RESTler* will treat unresolved dependencies in consumer parameters as `restler_fuzzable` string primitives. During fuzzing, *RESTler* reports each *bug*, currently defined as a 500 HTTP status code (500 “Internal Server Error”) received after executing a request sequence, as soon as it is found.

B. Current Limitations

Currently, *RESTler* does not support requests for API endpoints with server-side redirects (e.g., 301 “Moved Perma-

nently”, 303 “See Other”, and 307 “Temporary Redirect”). Furthermore, *RESTler* currently can only find bugs defined as unexpected HTTP status codes. Such a simple test oracle cannot detect vulnerabilities that are not visible through HTTP status codes (e.g., “Information Exposure” and others). Despite these limitations, *RESTler* has already found confirmed bugs in a production-scale open-source application and in several Microsoft Azure and Office365 services, as will be discussed in Sections VI and VII.

V. EVALUATION

We present experimental results obtained with *RESTler* that answer the following questions:

- Q1:** Are both inferring dependencies among request types and analyzing dynamic feedback necessary for effective automated REST API fuzzing? (Section V-B)
- Q2:** Are tests generated by *RESTler* exercising deeper service-side logic as sequence length increases? (Section V-C)
- Q3:** How do the three search strategies implemented in *RESTler* compare across various APIs? (Section V-D)

We answer the first question (Q1) using a simple blog posts service with a REST API. We answer (Q2), and (Q3) using GitLab, an open-source, production-scale¹ web service for self-hosted Git. We conclude the evaluation by discussing in Section V-E how to bucketize (i.e., group together) the numerous bugs that can be reported by *RESTler* in order to facilitate their analysis.

A. Experimental Setup

Blog Posts Service. We answer (Q1) using a simple blog posts service, written in 189 lines of python code using the Flask web framework [12]. Its functionality is exposed over a REST API with a Swagger specification shown in Figure 1. The API contains five request types: (i) GET on `/posts`: returns all blog posts currently registered; (ii) POST on `/posts`: creates a new blog post (body: the text of the blog post); (iii) DELETE `/posts/id`: deletes a blog post; (iv) GET `posts/id`: returns the body and the checksum of an individual blog post; and (v) PUT `/posts/id`: updates the contents of a blog post (body: the new text of the blog post and the checksum of the older version of the blog post’s text).

To model an imaginary subtle bug, at every update of a blog post (PUT request with body text and checksum) the service checks if the checksum provided in the request matches the recorded checksum for the current blog post, and if it does, an uncaught exception is raised. Thus, this bug will be triggered and detected only if dependencies on dynamic objects shared across requests are taken into account during test generation.

GitLab. We answer (Q2) and (Q3) using GitLab, an open-source web service for self-hosted Git. GitLab’s back-end is written in over 376K lines of ruby code using ruby-on-rails [35] and its functionality is exposed through a REST

¹GitLab [13] is used by more than 100,000 organizations, has millions of users, and has currently a 2/3 market share of the self-hosted Git market [20].

API [14]. For our deployment, we apply the following configuration settings: we use Nginx to proxypass the Unicorn web server and configure 15 Unicorn workers limited to up to 2GB of physical memory; we use PostgreSQL for persistent storage configured with a pool of 10 workers; we use GitLab’s default configuration for sidekiq queues and redis workers. According to GitLab’s deployment recommendations, such configuration should scale up to 4,000 concurrent users [15].

Fuzzing Dictionaries. For the experiments in this section, we use the following dictionaries for fuzzable primitives types: *string* has possible values “sampleString” and “” (empty string); *integer* has possible values “0” and “1”; *boolean* has possible values “true” and “false”.

All experiments were run on Ubuntu 16.04 Microsoft Azure VMs configured with eight Intel(R) Xeon(R) E5-2673 v3 @ 2.40GHz CPU cores and 56GB of physical memory.

B. Techniques for Effective REST API Fuzzing

In this section, we report results with our blog posts service to determine whether both (1) inferring dependencies among request types and (2) analyzing dynamic feedback are necessary for effective automated REST API fuzzing (Q1). We choose a simple service in order to clearly measure and interpret the testing capabilities of the two core techniques being evaluated. Those capabilities are evaluated by measuring service code coverage and client-visible HTTP status codes.

Specifically, we compare results obtained when exhaustively generating all possible request sequences of length up to three, with three different test-generation algorithms:

- 1) *RESTler* ignores dependencies among request types and treats dynamic objects – such as post *id* and *checksum* – as fuzzable primitive type *string* objects, while still analyzing dynamic feedback.
- 2) *RESTler* ignores service-side dynamic feedback and does not eliminate invalid sequences during the search, but still infers dependencies among request types and generates request sequences satisfying those.
- 3) *RESTler* uses the algorithm of Figure 3 using both dependencies among request types and dynamic feedback.

Figure 4 shows the number of tests, *i.e.*, request sequences, up to maximum length 3, generated by each of these three algorithms, from left to right. The top plots show the cumulative code coverage measured in lines of code over time, as well as when the sequence length increases. The bottom plots show the cumulative number of HTTP status codes received.

Code Coverage. First, we observe that without considering dependencies among request types (Figure 4, top left), code coverage is limited to up to 130 lines and there is no increase over time, despite increasing the length of request sequences. This illustrates the limitations of using a naive approach to test a service where values of dynamic objects like *id* and *checksum* cannot be randomly guessed or picked among values in a small predefined dictionary. In contrast, by inferring dependencies among requests and by processing service responses *RESTler* achieves an increase in code coverage up to 150 lines of code (Figure 4, top center and right).

Second, we see that without considering dynamic feedback to prune invalid request sequences in the search space (Figure 4, top center) the number of tests generated grows quickly, even for a simple API. Specifically, without considering dynamic feedback (Figure 4, top center), *RESTler* produces more than 4,600 tests that take 1,750 seconds and cover about 150 lines of code. In contrast, by considering dynamic feedback (Figure 4, top right), the state space is significantly reduced and *RESTler* achieves the same code coverage with less than 800 test cases and only 179 seconds.

HTTP Status Codes. We make two observations. First, focusing on 40X status codes, we notice a high number of 40X responses when ignoring dynamic feedback (Figure 4, bottom center). This indicates that without considering service-side dynamic feedback, the number of possible invalid request sequences grows quickly. In contrast, considering dynamic feedback dramatically decreases the percentage of 40X status codes from 60% to 26% without using dependencies among request types (Figure 4 bottom left) and to 20% with using dependencies among request types (Figure 4, bottom right). Moreover, when using dependencies among request types (Figure 4, bottom right), we observe the highest percentage of 20X status codes (approximately 80%), indicating that *RESTler* then exercises a larger part of the service logic – also confirmed by coverage data (Figure 4, top right).

Second, when ignoring dependencies among request types, we see that no 500 status codes are detected (Figure 4, bottom left), while *RESTler* finds a handful of 500 status codes when using dependencies among request types (see (Figure 4, bottom left and bottom right)). These 500 responses are triggered by the unhandled exception we planted in our blog posts service after a PUT blog update request with a checksum matching the previous blog post’s body (see Section V-A). When ignoring dependencies among request types, *RESTler* misses this bug (Figure 4, bottom left). In contrast, when analyzing dependencies across request types and using the checksum returned by a previous GET `/posts/id` request in a subsequent PUT `/posts/id` update request with the same *id*, *RESTler* does trigger the bug. Furthermore, when additionally using dynamic feedback, the search space is pruned while preserving this bug, which is then found with the least number of tests (Figure 4, bottom right).

Overall, these experiments illustrate the complementarity between utilizing dependencies among request types and using dynamic feedback, and show that both are needed for effective REST API fuzzing.

C. Deeper Service Exploration

In this section, we use GitLab to determine whether tests generated by *RESTler* exercise deeper service-side logic as sequence length increases (Q2). We perform individual experiments on six groups of GitLab APIs related to usual operations with commits, branches, issues and notes, repositories and repository files, groups and group membership, and projects.

Table I shows the total number of requests in each of the six target API groups and presents experimental results obtained